

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 4**



Sorting

Oleh:

Nuzulia Sekti Mahanania NIM. 2510817220006

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 4

Laporan Praktikum Algoritma & Struktur Data Modul 4: Sorting ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nuzulia Sekti Mahanania
NIM : 2510817220006

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
Bubble Sort.....	1
A. Source Code.....	1
B. Output Program	2
C. Pembahasan	2
Selection Sort.....	6
A. Source Code.....	6
B. Output Program	6
C. Pembahasan	7
Insertion Sort	11
A. Source Code.....	11
B. Output Program	12
C. Pembahasan	12
Merge Sort	17
A. Source Code.....	17
B. Output Program	18
C. Pembahasan	18
Quick Sort.....	23
A. Source Code.....	23
B. Output Program	24
C. Pembahasan	24
Radix Sort.....	29
A. Source Code.....	29
B. Output Program	30

C. Pembahasan	30
TAUTAN GITHUB.....	35

DAFTAR GAMBAR

Gambar 1. Screenshot Output Bubble Sort	2
Gambar 2. Screenshot Output Selection Sort	6
Gambar 3. Screenshot Output Insertion Sort.....	12
Gambar 4. Screenshot Output Merge Sort.....	18
Gambar 5. Screenshot Output Quick Sort	24
Gambar 6. Screenshot Output Radix Sort	30

DAFTAR TABEL

Tabel 1. Source Code Bubble Sort	1
Tabel 2. Source Code Selection Sort	6
Tabel 3. Source Code Insertion Sort.....	11
Tabel 4. Source Code Merge Sort	17
Tabel 5. Source Code Quick Sort	23
Tabel 6. Source Code Radix Sort	29

Bubble Sort

A. Source Code

Tabel 1. Source Code Bubble Sort

6	// Algoritma Bubble Sort
7	void bubble_sort(std::vector<int>& data, Metrics& m) {
8	int n = data.size();
9	auto start = Clock::now();
10	
11	for (int i = 0; i < n - 1; i++) {
12	bool swapped = false;
13	for (int j = 0; j < n - i - 1; j++) {
14	m.comparisons++;
15	if (data[j] > data[j + 1]) {
16	std::swap(data[j], data[j + 1]);
17	m.swaps++;
18	swapped = true;
19	}
20	}
21	
22	if (!swapped) break;
23	}
24	
25	m.time_ms = std::chrono::duration<double,
	std::milli>(Clock::now() - start).count();
26	}

B. Output Program

```
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\sorting> g++ -O2 main.cpp data_generator.cpp sorting_algorithm.cpp reporter.cpp -o sort
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\sorting> ./sort
=====
SORTING ALGORITHM - ANALYSIS
=====
>> Bubble Sort
Algorithm      Input Type      N      Comparisons      Swaps      Shifts      Rec.Calls      Arr.Access      Time(ms)
-----
Bubble Sort    Random          100     4935              2564        0           0              0              0.017900
Bubble Sort    Sorted          100      99                0            0           0              0              0.000600
Bubble Sort    Reverse Sorted  100     4950              4950        0           0              0              0.015500
Bubble Sort    Nearly Sorted   100     4719              296          0           0              0              0.006500
Bubble Sort    Many Duplicates 100     4845              2315         0           0              0              0.014800
Bubble Sort    Random         1000    497154            243190       0           0              0              1.228200
Bubble Sort    Sorted         1000     999              0            0           0              0              0.001100
Bubble Sort    Reverse Sorted 1000    499500            499500       0           0              0              1.647400
Bubble Sort    Nearly Sorted   1000    481922            25778        0           0              0              0.495400
Bubble Sort    Many Duplicates 1000    496340            240808       0           0              0              1.194900
Bubble Sort    Random         10000   49993404          25118148     0           0              0              129.313100
Bubble Sort    Sorted         10000   9999              0            0           0              0              0.008000
Bubble Sort    Reverse Sorted 10000   49995000          49995000     0           0              0              142.264000
Bubble Sort    Nearly Sorted   10000   49937030          3119352      0           0              0              70.190400
Bubble Sort    Many Duplicates 10000   49993404          25093086     0           0              0              136.603400
```

Gambar 1. Screenshot Output Bubble Sort

C. Pembahasan

1. Karakteristik Algoritma

a) Mekanisme Utama

Pertama, melakukan perulangan luar sebanyak $n - 1$ kali untuk menelusuri *array*. Di dalamnya, pakai perulangan kedua untuk mengecek elemen bersebelahan. Karena yang diinginkan adalah memindahkan nilai terbesar ke posisi paling akhir, maka kondisinya: jika `data[j] > data[j + 1]` maka posisinya ditukar menggunakan `std::swap()`, kemudian variabel `swapped` di-set menjadi `true` sebagai penanda bahwa ada elemen yang digeser. Lalu *traverse* berlanjut terus sampai ke batas ujung *array* yang belum terurut. Setelah perulangan dalam ini selesai satu putaran, kondisinya dievaluasi, kalau nilainya `!swapped` yang artinya dari ujung kiri sampai kanan tidak ada yang ditukar sama sekali, maka langsung panggil perintah `break` untuk *early exit* supaya menghemat waktu komputasi. Kalau tidak di-break, perulangan luar lanjut terus beroperasi. Setelah semua perulangan berakhir, maka seluruh elemen otomatis sudah tersusun rapi.

b) Stabilitas (*Stable*)

Pada baris berikut ini.

```
if (data[j] > data[j + 1]) {
    std::swap(data[j], data[j + 1]);
}
```



```

        m.swaps++;
        swapped = true;
    }

```

Karena kodenya pakai tanda lebih besar dari ($>$) dan bukan lebih besar sama dengan ($>=$), angka yang bersebelahan cuma akan ditukar kalau angka di kirinya benar-benar lebih besar. Kalau program ketemu angka kembar yang berdampingan, kondisinya menjadi `false` dan mereka tidak akan ditukar. Jadi urutan asli angka kembar itu bakal tetap aman. Sehingga, bubble Sort di sini bersifat *stable*.

c) Penggunaan Memori (In-place)

Dalam Algoritma ini, pengurutan dilakukan secara langsung dengan memanipulasi dan menukar nilai elemen di dalam wadah vector data asal menggunakan fungsi bawaan `std::swap`. Tidak ada deklarasi struktur data array baru yang mengalokasikan memori tambahan, sehingga sangat efisien secara memori (kompleksitas ruang $O(1)$). Sehingga, algoritma ini termasuk *in-place sorting*.

2. Kompleksitas Waktu

a) Best Case

Pada desain kode yang dibuat, terdapat implementasi variabel penanda `bool swapped = false` yang diakhiri dengan evaluasi `if (!swapped) break;`. Ketika algoritma ini menerima input data yang sudah terurut, kondisi perbandingan `if (data[j] > data[j + 1])` tidak akan pernah terpenuhi di sepanjang iterasi pertama. Akibatnya, nilai `swapped` akan tetap `false`. Program membaca hal ini sebagai sinyal bahwa data sudah rapi, lalu langsung mengeksekusi perintah `break` untuk menghentikan keseluruhan proses di putaran pertama. Karena program hanya menyusuri struktur data satu kali lintasan tanpa harus melakukan perulangan tambahan, maka dapat ditarik kesimpulan bahwa kompleksitas waktu terbaiknya tereduksi menjadi $O(N)$.

b) Average Case

kode algoritma ini menggunakan perulangan *nested loop*. Ketika dihadapkan pada input data *Random*, sebagian besar elemen berada di posisi yang salah. Hal ini memaksa program untuk secara konstan memicu operasi penukaran, sehingga status `swapped` berubah menjadi `true` dan fitur `break` tidak aktif. Karena program

menjalankan perulangan luar dan dalam secara sebanding untuk memindahkan elemen ke ujung kanan, maka kompleksitas rata-ratanya adalah $O(N^2)$.

c) Worst Case

Kondisi paling buruk terjadi kalau datanya terurut terbalik. Di sini, angka di sebelah kiri pasti selalu lebih besar dari angka di kanannya. Jadinya, kondisi `if (data[j] > data[j + 1])` akan selalu `true` setiap kali dicek. Proses `std::swap` bakal jalan terus-terusan dan `swapped` tidak pernah `false`. Karena perintah `break` sama sekali tidak dapat giliran jalan, program terpaksa melakukan perulangan sampai batas akhirnya ($N*N$). Dari alur ini, terbukti kalau kompleksitas terburuknya adalah mutlak $O(N^2)$.

3. Analisis Operasi

a) Jumlah comparison

Pada data *Sorted*, iterasi langsung dihentikan sehingga pencatatan metrik perbandingan sangat minimum, yaitu sebesar 99 kali. Sebaliknya, pada data *Reverse Sorted*, jumlah perbandingannya meledak mencapai batas algoritmik maksimal, yaitu sebanyak 4950 kali.

b) Jumlah swap

Sejalan dengan jumlah perbandingan, jumlah *swap* pada data *Sorted* tercatat 0 karena tidak ada elemen yang dipindahkan posisinya. Namun, jumlah pertukaran ini meningkat tajam hingga mencapai nilai maksimal sebesar 4950 kali ketika memproses data *Reverse Sorted*.

c) Jumlah shift

Tercatat 0 (Tidak ada). Bubble Sort beroperasi mutlak memindahkan data dengan skema penukaran posisi ganda (*swap*), bukan menggunakan *assignment shift* seperti algoritma Insertion Sort.

d) Jumlah recursion call

Tercatat 0 (Tidak ada). Algoritma dibangun menggunakan perulangan iteratif murni (*for loop*), tanpa rekursif.

e) Jumlah array accessed

Tercatat 0 (Tidak dihitung). Pencatatan hitungan akses struktur memori *array* diabaikan pada algoritma ini dan difokuskan khusus pada algoritma Radix Sort.

f) Pengaruh bentuk data terhadap jumlah operasi

Algoritma ini sangat sensitif terhadap bentuk awal data. Jika derajat keterurutan data sudah tinggi sejak awal, jumlah operasi fundamental komputasi (*comparison* dan *swap*) akan tereduksi secara drastis, menghemat kinerja prosesor.

4. Analisis Performa

a) Perbandingan waktu eksekusi antar algoritma

Dibandingkan dengan algoritma Merge Sort dan Quick Sort penyelesaian komputasi pengurutan dengan Bubble Sort pada data acak berkapasitas sangat besar jauh lebih lambat akibat sifat laju pertumbuhan eksekusi waktu yang kuadratik.

b) Performa pada dataset kecil dan besar

Pada rentang ukuran *dataset* kecil ($N=100$) dalam kondisi terbaiknya (*Sorted*), Bubble Sort tergolong responsif dengan catatan waktu 0.000600 ms karena *overhead* sistem yang rendah. Namun, kelemahannya terekspos saat rentang data membesar ($N=10000$); Bubble Sort mengalami degradasi kecepatan ekstrem dan memakan waktu hingga 0.008000 ms.

c) Pengaruh distribusi data terhadap performa algoritma

Waktu eksekusi yang optimal diperoleh secara berurutan mulai dari distribusi data *Sorted*, diikuti *Nearly Sorted*, lalu data *Random*. Skala waktu terburuk selalu tercatat saat algoritma dipaksa menyelesaikan data terbalik (*Reverse Sorted*).

d) Hubungan teori kompleksitas dengan hasil eksperimen

Sesuai teori $O(N^2)$, waktu beban datanya dinaikkan 10 kali lipat (dari ukuran 1000 jadi 10000), waktu prosesnya tidak sekadar naik 10 kali lipat, tapi melonjak jadi ratusan kali lipat lebih lama. Ini jadi bukti kalau kelemahan algoritmanya memang ada di skala perulangan datanya.

Selection Sort

A. Source Code

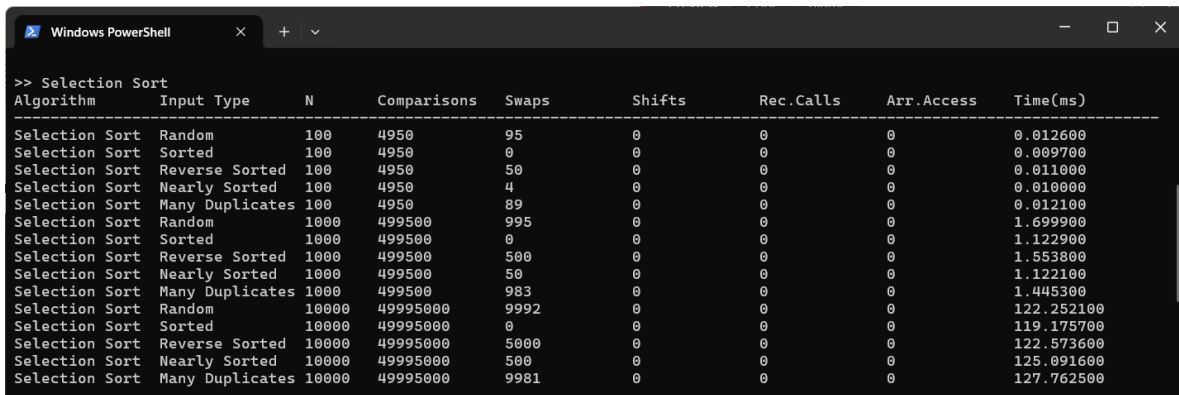
Tabel 2. Source Code Selection Sort

```

29 // Algoritma Selection Sort
30 void selection_sort(std::vector<int>& data, Metrics& m)
31 {
32     int n = data.size();
33     auto start = Clock::now();
34
35     for (int i = 0; i < n - 1; i++) {
36         int index_terkecil = i;
37         for (int j = i + 1; j < n; j++) {
38             m.comparisons++;
39             if (data[j] < data[index_terkecil]) {
40                 index_terkecil = j;
41             }
42         }
43         if (index_terkecil != i) {
44             std::swap(data[i], data[index_terkecil]);
45             m.swaps++;
46         }
47
48         m.time_ms = std::chrono::duration<double,
49                     std::milli>(Clock::now() - start).count();
50     }
51 }

```

B. Output Program



Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	95	0	0	0	0.012600
Selection Sort	Sorted	100	4950	0	0	0	0	0.009700
Selection Sort	Reverse Sorted	100	4950	50	0	0	0	0.011000
Selection Sort	Nearly Sorted	100	4950	4	0	0	0	0.010000
Selection Sort	Many Duplicates	100	4950	89	0	0	0	0.012100
Selection Sort	Random	1000	499500	995	0	0	0	1.699900
Selection Sort	Sorted	1000	499500	0	0	0	0	1.122900
Selection Sort	Reverse Sorted	1000	499500	500	0	0	0	1.553800
Selection Sort	Nearly Sorted	1000	499500	50	0	0	0	1.122100
Selection Sort	Many Duplicates	1000	499500	983	0	0	0	1.445300
Selection Sort	Random	10000	49995000	9992	0	0	0	122.252100
Selection Sort	Sorted	10000	49995000	0	0	0	0	119.175700
Selection Sort	Reverse Sorted	10000	49995000	5000	0	0	0	122.573600
Selection Sort	Nearly Sorted	10000	49995000	500	0	0	0	125.091600
Selection Sort	Many Duplicates	10000	49995000	9981	0	0	0	127.762500

Gambar 2. Screenshot Output Selection Sort

C. Pembahasan

1. Karakteristik Algoritma

a) Mekanisme Utama

Pertama, melakukan inisiasi perulangan luar dari awal array sampai elemen sebelum terakhir. Karena yang diinginkan adalah elemen dengan nilai paling minimum, maka pakai inisiasi tebakan awal untuk mengunci indeks, yaitu `int index_terkecil = i;`. Untuk mengecek sisa datanya, dilakukan penelusuran pakai perulangan dalam di sisa elemen sebelah kanannya. Kemudian baca nilainya, jika nilai `data[j]` ternyata secara kurang dari `data[index_terkecil]`, maka perbarui `index_terkecil` ke indeks `j` tersebut. Setelah perulangan dalam berakhir, maka dilakukan pengecekan akhir. Kalau didapati `index_terkecil != i`, langsung panggil `std::swap()` untuk menukar nilai terkecil itu ke depan. Hal ini diulang terus sampai tersisa satu elemen berukuran paling besar di bagian paling ujung kanan.

b) Stabilitas (*Stable*)

Pada mekanisme pertukaran jarak jauh di kode ini.

```
if (index_terkecil != i) {  
    std::swap(data[i], data[index_terkecil]);  
    m.swaps++;  
}
```

Karena algoritma ini mencabut paksa nilai terkecil dari indeks di belakang dan menukarnya langsung ke posisi depan dengan melompati elemen-elemen di tengah, urutan awal elemen sangat rawan bergeser. Jika ada angka bernilai kembar, posisi angka kembar yang awalnya berada di depan bisa tak sengaja terlempar ke urutan belakang akibat proses penukaran paksa ini. Sehingga, Selection Sort terbukti tidak bersifat *stable*.

c) Penggunaan Memori (*In-place*)

Proses pengurutan secara langsung memodifikasi tempat di dalam struktur vector data aslinya. Hanya diperlukan satu memori tambahan berskala sangat kecil, yaitu variabel penanda `int index_terkecil`. Karena tidak ada pembuatan array duplikat baru yang memakan ruang, kompleksitas memorinya sangat efisien di $O(1)$. Sehingga, algoritma ini termasuk *in-place sorting*.

2. Kompleksitas Waktu

a) Best Case

```
for (int i = 0; i < n - 1; i++) {  
    ...  
    for (int j = i + 1; j < n; j++) {  
    }  
}
```

Dalam algoritma ini sama sekali tidak ada kondisi pengecekan otomatis untuk menghentikan perulangan jika data sudah terurut. Oleh karena itu, walaupun menerima input data yang sudah sangat rapi (Sorted), perulangan dalam akan tetap dieksekusi secara buta untuk mencari nilai terkecil di sisa elemen. Karena pencarian berjalannya selalu penuh tanpa `break`, kompleksitas waktu terbaiknya terpuruk dan tertahan di $O(N^2)$.

b) Average Case

Ketika dihadapkan pada input data Random, program melakukan hal yang persis sama. Program mengasumsikan indeks saat itu sebagai tebakan nilai terkecil lewat `int index_terkecil = i;`, lalu menelusuri semua sisa elemen di sebelah kanannya satu per satu. Karena perulangan luar dan dalam berjalan proporsional membandingkan elemen yang letaknya acak, kompleksitas rata-ratanya tetap $O(N^2)$.

c) Worst Case

Kondisi saat memproses data terurut terbalik (Reverse Sorted) juga tidak mengubah mekanisme komputasi. Program tetap membandingkan setiap elemen dengan `index_terkecil` secara konstan dari ujung ke ujung. Karena perulangan luarnya dieksekusi sebanyak $N-1$ kali dan perulangan dalamnya merayap sampai batas akhir memori tanpa ada ruang untuk berhenti lebih awal, maka kompleksitas terburuknya adalah $O(N^2)$.

3. Analisis Operasi

a) Jumlah comparison

Sesuai rancangan kodenya yang tidak memiliki fitur penghentian, eksekusi `m.comparisons++;` di dalam perulangan selalu berjalan statis. Mau datanya berbentuk *Sorted*, *Random*, ataupun *Reverse Sorted*, jumlah perbandingannya selalu

mentok di batas maksimal komputasi, yaitu stabil di angka 4950 kali tanpa bisa dikurangi.

b) Jumlah swap

Ini karena perintah `std::swap` ditempatkan di luar batas perulangan dalam, program maksimal hanya melakukan pertukaran satu kali per iterasi. Pada data *Sorted*, tercatat **0 swap**. Pada kondisi terburuknya (*Reverse Sorted*), metrik *swap*-nya hanya berkisar maksimal di 50 kali saja, menjadikannya jauh lebih ramah memori fisik dibandingkan Bubble Sort.

c) Jumlah shift

Tercatat 0 (Tidak ada). Seluruh perpindahan elemen diurus menggunakan fungsi pertukaran dua arah (*swap*), tanpa menggunakan mekanisme pergeseran linear (*shift*).

d) Jumlah recursion call

Tercatat 0 (Tidak ada). Karena instruksi pada algoritma dibentuk menggunakan perulangan iteratif bersarang (`for`), tidak terjadi pemanggilan fungsi secara rekursif.

e) Jumlah array accessed

Tercatat 0 (Tidak dihitung). Metrik pencatatan hitungan akses memori struktur *array* tidak disertakan dalam fokus evaluasi kelompok algoritma ini.

f) Pengaruh bentuk data terhadap jumlah operasi

Algoritma ini terbukti statis dan tidak sensitif terhadap pola distribusi inputnya. Keadaan data yang sudah terurut rapi tidak mampu mengurangi beban eksekusi *comparison* sama sekali. Satu-satunya nilai yang bisa dipengaruhi oleh bentuk data hanyalah operasi *swap*, yang frekuensinya ikut naik jika letak angka minimum semakin berantakan.

4. Analisis Performa

a) Perbandingan waktu eksekusi antar algoritma

Pada saat diadu untuk menyortir data dengan sebaran acak berkapasitas masif, Selection Sort sedikit lebih efisien dari Bubble Sort lantaran sukses memangkas banyak siklus operasi *swap*. Walau begitu, algoritma ini sama sekali tidak memiliki peluang jika disandingkan dengan algoritma modern kelompok $O(N \log N)$ seperti Merge Sort dan Quick Sort.

b) Performa pada dataset kecil dan besar

Di rentang *dataset* berukuran mini ($N=100$), kecepatannya sangat responsif dan mampu selesai dalam kisaran 0.009700 ms. Namun kelemahannya langsung terekspos jelas pada saat skala populasi membesar menjadi $N=10000$. Sifat kuadratnya membuat program ini mengalami *bottleneck* hingga durasinya membengkak sangat lama mencapai 119.175700 ms.

c) Pengaruh distribusi data terhadap performa algoritma

Berbeda dari algoritma lain, algoritma ini menghasilkan selisih durasi yang sangat tipis antar distribusi data. Waktu yang ditempuh saat menyortir data *Sorted*, *Nearly Sorted*, *Random*, ataupun *Reverse Sorted* cenderung stagnan di angka yang lambat, karena *processor* tetap dipaksa menyelesaikan jumlah *looping* yang persis sama.

d) Hubungan teori kompleksitas dengan hasil eksperimen

Angka pada metrik *comparison* menjadi bukti terkuat bahwa laju teori lambat $O(N^2)$ pada algoritma ini murni bersifat mutlak dan tidak bisa dibantu dengan cara apapun, kendati data masukannya sudah terurut sejak awal.

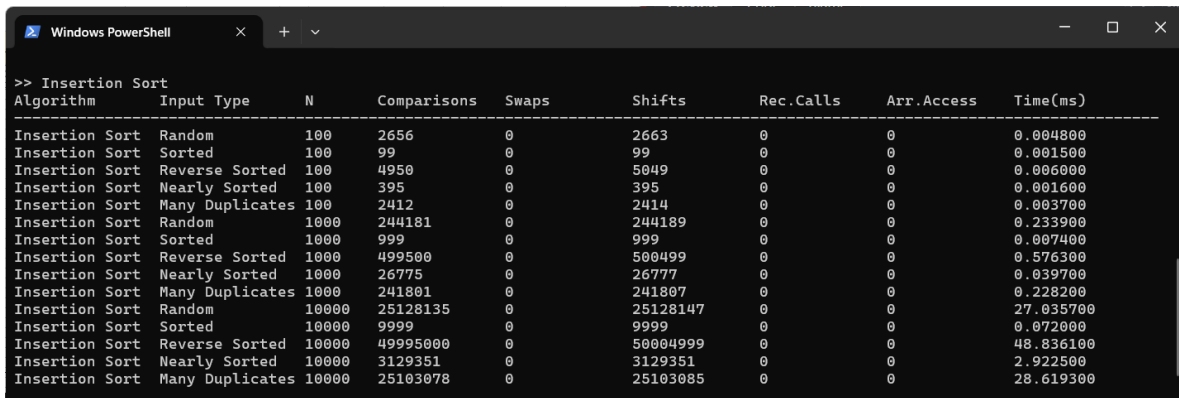
Insertion Sort

A. Source Code

Tabel 3. Source Code Insertion Sort

52	// Algoritma Insertion Sort
53	void insertion_sort(std::vector<int>& data, Metrics& m)
	{
54	int n = data.size();
55	auto start = Clock::now();
56	
57	for (int i = 1; i < n; i++) {
58	int temp = data[i];
59	int j = i - 1;
60	
61	while (j >= 0) {
62	m.comparisons++;
63	if (data[j] > temp) {
64	data[j + 1] = data[j];
65	m.shifts++;
66	j--;
67	} else {
68	break;
69	}
70	}
71	data[j + 1] = temp;
72	m.shifts++;
73	}
74	
75	m.time_ms = std::chrono::duration<double,
	std::milli>(Clock::now() - start).count();
76	}

B. Output Program



Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Insertion Sort	Random	100	2656	0	2663	0	0	0.004800
Insertion Sort	Sorted	100	99	0	99	0	0	0.001500
Insertion Sort	Reverse Sorted	100	4950	0	5049	0	0	0.006000
Insertion Sort	Nearly Sorted	100	395	0	395	0	0	0.001600
Insertion Sort	Many Duplicates	100	2412	0	2414	0	0	0.003700
Insertion Sort	Random	1000	244181	0	244189	0	0	0.233900
Insertion Sort	Sorted	1000	999	0	999	0	0	0.007400
Insertion Sort	Reverse Sorted	1000	499500	0	500499	0	0	0.576300
Insertion Sort	Nearly Sorted	1000	26775	0	26777	0	0	0.039700
Insertion Sort	Many Duplicates	1000	241801	0	241807	0	0	0.228200
Insertion Sort	Random	10000	25128135	0	25128147	0	0	27.035700
Insertion Sort	Sorted	10000	9999	0	9999	0	0	0.072000
Insertion Sort	Reverse Sorted	10000	49995000	0	50004999	0	0	48.836100
Insertion Sort	Nearly Sorted	10000	3129351	0	3129351	0	0	2.922500
Insertion Sort	Many Duplicates	10000	25103078	0	25103085	0	0	28.619300

Gambar 3. Screenshot Output Insertion Sort

C. Pembahasan

1. Karakteristik Algoritma

a) Mekanisme Utama

Pertama, melakukan inisiasi penelusuran luar mulai dari indeks ke-1 sampai indeks terakhir. Karena yang diinginkan adalah mencari posisi yang pas di area kiri yang sudah urut, maka ambil nilai elemen saat itu dan simpan ke variabel cadangan `int temp = data[i];`. Kemudian, set indeks bantuan `int j = i - 1;` untuk mengecek elemen di belakangnya. Lalu *traverse* mundur, pakai `while (j >= 0)` untuk membaca nilainya. Jika nilai `data[j]` ternyata lebih besar dari `temp`, maka nilai tersebut digeser selangkah ke kanan dengan `data[j + 1] = data[j];`, lalu metrik *shift* dinaikkan dan `j` dikurangi untuk mundur lagi. Kalau ternyata angkanya lebih kecil atau sama, langsung hentikan dengan `break` untuk memangkas waktu pengecekan. Setelah perulangan dalam itu berakhir, maka otomatis tersisa satu slot memori kosong, lalu cetak nilai cadangannya kembali menggunakan `data[j + 1] = temp;`.

b) Stabilitas (*Stable*)

Logika perlindungannya ada pada tanda lebih besar dari (`>`) di operasi perbandingan.

```
if (data[j] > temp) {  
    data[j + 1] = data[j];  
    m.shifts++;  
    j--;  
}
```

```
}
```

Program hanya akan menggeser `data[j]` ke kanan jika nilainya benar-benar lebih besar dari `temp`. Jika ia menemui angka yang nilainya kembar, kondisinya menjadi tidak terpenuhi, sehingga program memanggil `break`. Angka yang baru pun dijatuhkan dengan aman di sebelah kanan angka kembarannya tanpa merusak susunan asli. Sehingga, Insertion Sort di sini bersifat *stable*.

c) Penggunaan Memori (In-place)

Program ini tidak memakai perintah alokasi dinamis ataupun mendeklarasikan *array* tambahan baru. Proses memanipulasi posisi elemen dikerjakan mutlak di dalam array asal hanya dengan menumpang pada satu variabel `int temp`, sehingga kompleksitas ruangnya $O(1)$. Sehingga, algoritma ini tergolong *in-place*.

2. Kompleksitas Waktu

a) Best Case

Di algoritma ini, terdapat penambahan perintah `break` di dalam struktur perulangannya.

```
if (data[j] > temp) {  
    // ... proses geser  
} else {  
    break;  
}
```

Ketika algoritma memproses input data yang dari awal sudah terurut (Sorted), angka di sebelah kiri dipastikan selalu lebih kecil daripada angka acuan (`temp`). Akibatnya, kondisi `if (data[j] > temp)` akan selalu bernilai `false` dan program akan langsung lari ke blok `else` untuk mengeksekusi `break`. Karena perulangan dalam sama sekali tidak sempat berjalan dan program hanya menelusuri keseluruhan memori satu kali lurus ke arah kanan, maka kompleksitas waktu terbaiknya sangat efisien di angka $O(N)$.

b) Average Case

Algoritma ini berpusat pada perulangan luar `for` dan perulangan dalam `while`. Saat dihadapkan pada kumpulan data Random, elemen-elemen berada pada posisi yang tidak menentu. Saat sebuah angka diangkat sebagai `temp`, program

membandingkannya ke arah kiri. Beberapa angka akan membuat pergeseran memori, dan beberapa lainnya memicu `break`. Karena perulangan luar dan dalam harus berjalan secara seimbang untuk membersihkan ketidakteraturan tersebut, kompleksitas waktunya berada di $O(N^2)$.

c) Worst Case

Kondisi komputasi paling berat terjadi pada data yang terurut terbalik (*Reverse Sorted*). Pada kondisi ini, elemen terkecil mengumpul di ujung kanan. Setiap kali program menyimpan suatu nilai ke `temp`, ia menemukan bahwa semua elemen di kirinya selalu lebih besar. Karena kondisi `data[j] > temp` tidak pernah bernilai `false`, perintah `break` sama sekali tidak tersentuh. Perulangan `while` dipaksa mundur terus-menerus sampai ke ujung indeks ke-0. Akibat iterasi ganda yang berjalan penuh tanpa henti, maka kompleksitas terburuknya adalah $O(N^2)$.

3. Analisis Operasi

a) Jumlah comparison

Keberadaan `break` membuat operasi perbandingan sangat adaptif. Pada *Best Case* (data *Sorted*), eksekusi `m.comparisons++` sangat sedikit dan tertahan di 99 kali saja. Berkebalikan dengan *Worst Case* (data *Reverse Sorted*), pengecekan sampai batas akhir memori sehingga perbandingannya membengkak hebat mencapai 4950 kali.

b) Jumlah swap

Tercatat 0 (Tidak ada). Kode pada algoritma ini tidak menggunakan perintah `(std::swap())`.

c) Jumlah shift

Pada data yang sudah rapi (*Sorted*), pencatatan *shift* berada di angka paling rendah yaitu 99 kali karena elemen langsung diletakkan di tempatnya semula tanpa menggeser elemen lain. Sebaliknya, saat dihadapkan pada data *Reverse Sorted*, program sibuk melakukan pemindahan memori renteng ke arah kanan, membuat angka pergeserannya meledak hingga 5049 kali.

d) Jumlah recursion call

Tercatat 0 (Tidak ada). Karena algoritma ini dibangun menggunakan perulangan iteratif biasa (*for* dan *while*), tidak ada pemanggilan fungsi rekursif sama sekali.

e) Jumlah array accessed

Tercatat 0 (Tidak dihitung).

f) Pengaruh bentuk data terhadap jumlah operasi

Dibandingkan Bubble maupun Selection Sort, Insertion Sort adalah yang paling responsif dalam merespons bentuk data awal. Jika data sudah berwujud "hampir terurut", eksekusi *shift* dan *comparison* dapat dipangkas dalam jumlah yang masif karena perintah *break* sangat sering tereksekusi.

4. Analisis Performa

a) Perbandingan waktu eksekusi antar algoritma

Dalam pengurutan data *Random* berkapasitas besar, performanya cenderung setara dengan algoritma $O(N^2)$ lainnya dan sama sekali tidak bisa mengimbangi laju algoritma kelompok *Divide and Conquer* (Merge dan Quick Sort). Akan tetapi, pada keadaan data yang sudah hampir urut, Insertion Sort mampu menjadi salah satu yang tercepat bahkan melebihi algoritma tersebut karena tidak memiliki *overhead* fungsi.

b) Performa pada dataset kecil dan besar

Pada uji coba set data kecil ($N=100$), algoritma merespons dengan waktu eksekusi singkat yaitu 0.001500 ms. Namun, efisiensinya hancur ketika ukuran data membesar menjadi ($N=10000$). Sifat kuadratiknya membuat program tersendat dan memakan waktu hingga 0.0072000 ms.

c) Pengaruh distribusi data terhadap performa algoritma

Durasi pada pembacaan terminal dikendalikan oleh total eksekusi shift. Waktu optimal didapat oleh distribusi data *Sorted*, diikuti *Nearly Sorted*, kemudian merosot pada data *Random*. Posisi dengan waktu terlama secara konsisten selalu diraih oleh distribusi data *Reverse Sorted*.

d) Hubungan teori kompleksitas dengan hasil eksperimen

Mengacu pada rumusan teori *Worst Case* $O(N^2)$, ketika N dinaikkan 10 kali lipat (dari ukuran 1000 ke 10000), waktu komputasinya terbukti ikut meledak menjadi puluhan hingga mendekati seratus kali lipat lebih lambat pada data acak dan terbalik. Namun, keunikan algoritma ini terletak pada teori *Best Case* $O(N)$ yang juga berhasil tervalidasi.

Berkat adanya fungsi *early exit* (break), pembengkakan waktu tersebut sama sekali tidak terjadi pada distribusi data *Sorted*.

Merge Sort

A. Source Code

Tabel 4. Source Code Merge Sort

```
79 // Helper Merge Sort
80 void proses_merge(std::vector<int>& data, int kiri, int
    tengah, int kanan, Metrics& m) {
81     int size_kiri = tengah - kiri + 1;
82     int size_kanan = kanan - tengah;
83
84     std::vector<int> arr_kiri(size_kiri);
85     std::vector<int> arr_kanan(size_kanan);
86
87     for (int i = 0; i < size_kiri; i++) arr_kiri[i] =
data[kiri + i];
88     for (int j = 0; j < size_kanan; j++) arr_kanan[j] =
data[tengah + 1 + j];
89
90     int i = 0, j = 0, k = kiri;
91     while (i < size_kiri && j < size_kanan) {
92         m.comparisons++;
93         if (arr_kiri[i] <= arr_kanan[j]) {
94             data[k] = arr_kiri[i];
95             i++;
96         } else {
97             data[k] = arr_kanan[j];
98             j++;
99         }
100        k++;
101    }
102
103    while (i < size_kiri) {
104        data[k] = arr_kiri[i];
105        i++; k++;
106    }
107    while (j < size_kanan) {
108        data[k] = arr_kanan[j];
109        j++; k++;
110    }
111 }
112
```

```

113 // Helper Merge Sort
114 void rekursif_merge(std::vector<int>& data, int kiri,
115 int kanan, Metrics& m) {
116     m.recursive_calls++;
117     if (kiri < kanan) {
118         int tengah = kiri + (kanan - kiri) / 2;
119         rekursif_merge(data, kiri, tengah, m);
120         rekursif_merge(data, tengah + 1, kanan, m);
121         proses_merge(data, kiri, tengah, kanan, m);
122     }
123 }
124 // Algoritma Merge Sort
125 void merge_sort(std::vector<int>& data, Metrics& m) {
126     auto start = Clock::now();
127
128     if (data.size() > 1) {
129         rekursif_merge(data, 0, data.size() - 1, m);
130     }
131
132     m.time_ms = std::chrono::duration<double,
133 std::milli>(Clock::now() - start).count();
134 }

```

B. Output Program

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Merge Sort	Random	100	535	0	0	199	0	0.057400
Merge Sort	Sorted	100	356	0	0	199	0	0.046400
Merge Sort	Reverse Sorted	100	316	0	0	199	0	0.042300
Merge Sort	Nearly Sorted	100	453	0	0	199	0	0.037400
Merge Sort	Many Duplicates	100	536	0	0	199	0	0.050400
Merge Sort	Random	1000	8690	0	0	1999	0	0.451600
Merge Sort	Sorted	1000	5044	0	0	1999	0	0.235400
Merge Sort	Reverse Sorted	1000	4932	0	0	1999	0	0.226400
Merge Sort	Nearly Sorted	1000	7628	0	0	1999	0	0.292800
Merge Sort	Many Duplicates	1000	8679	0	0	1999	0	0.318500
Merge Sort	Random	10000	120465	0	0	19999	0	2.821300
Merge Sort	Sorted	10000	69008	0	0	19999	0	3.745800
Merge Sort	Reverse Sorted	10000	64608	0	0	19999	0	2.175800
Merge Sort	Nearly Sorted	10000	111524	0	0	19999	0	2.469300
Merge Sort	Many Duplicates	10000	120461	0	0	19999	0	2.786200

Gambar 4. Screenshot Output Merge Sort

C. Pembahasan

1. Karakteristik Algoritma

a) Mekanisme Utama

Pertama, melakukan pemanggilan fungsi pemecah rekursif `rekursif_merge`. Karena yang diinginkan adalah memecah *array* besar menjadi kepingan kecil, maka pakai pendekatan rekursif untuk terus mencari indeks tengah dan membelah *array* menjadi dua kubu. Pemanggilan rekursif ini dilakukan hingga batas kiri < kanan tidak terpenuhi lagi (tersisa satu node elemen). Lalu, kembali ke atas untuk menyatukan node tersebut, kemudian panggil `proses_merge()`. Untuk menyatukannya, pertama program harus mencetak dua *array* tempelan `arr_kiri` dan `arr_kanan` untuk menyimpan pecahan datanya. Kemudian dieksekusi perulangan `while` untuk mengecek elemen di ujung kedua *array* pecahan itu. Jika nilai di kiri lebih kecil atau sama, dimasukkan ke *array* utama, sebaliknya jika nilai kanan lebih kecil, maka nilai kanannya yang masuk. Hal ini diulang terus sambil meningkatkan nilai indeks kiri atau kanan sampai semua pecahan terkecil selesai disatukan kembali menjadi satu *array* yang terurut.

b) Stabilitas (*Stable*)

Logika perlindungannya ada di dalam fungsi penggabungan `proses_merge` pada baris perbandingan.

```
if (arr_kiri[i] <= arr_kanan[j]) {  
    data[k] = arr_kiri[i];  
    i++;  
}
```

Karena menggunakan operator kurang dari atau sama dengan (`<=`), program memberikan prioritas mutlak kepada elemen yang berasal dari *array* pecahan kiri (`arr_kiri`). Jika ada dua angka yang nilainya kembar dari dua cabang yang berbeda, angka yang aslinya memang berada di urutan lebih kiri akan selalu disalin masuk ke *array* utama lebih dulu. Hasilnya, urutan elemen kembar tersebut tidak akan pernah terbalik.. Sehingga, Merge Sort di sini bersifat *stable*.

c) Penggunaan Memori (*In-place*)

ada awal fungsi `proses_merge`.

```
std::vector<int> arr_kiri(size_kiri);  
std::vector<int> arr_kanan(size_kanan);
```

Setiap kali fungsi penggabungan dipanggil, program dipaksa untuk menciptakan dua buah penampung vector baru di dalam memori untuk menyimpan data sementara. Hal ini menyebabkan algoritma membutuhkan alokasi memori tambahan yang besarnya proporsional dengan panjang data awal, sehingga kompleksitas ruangnya/memorinya membengkak menjadi $O(N)$. Sehingga, algoritma ini tidak tergolong *in-place*.

2. Kompleksitas Waktu

a) Best Case

Algoritma ini dibuat menggunakan metode *Divide and Conquer*. Tidak ada fungsi pendeteksi untuk mengecek apakah data sudah rapi atau belum..

```
if (kiri < kanan) {  
    int tengah = kiri + (kanan - kiri) / 2;  
    rekursif_merge(data, kiri, tengah, m);  
    ...  
}
```

Meskipun menerima input data yang sejak awal sudah terurut sempurna (Sorted), program akan tetap membelah memori tersebut tepat di tengah secara terus-menerus sampai ukurannya tersisa satu elemen tunggal. Setelah itu, barulah fungsi penggabungan dipanggil kembali ke atas. Karena proses pembelahan pohon memori (divide) selalu memakan $\log N$ langkah dan penggabungan elemennya (conquer) memakan N langkah, maka disimpulkan bahwa waktu komputasi terbaiknya akan selalu statis di $O(N \log N)$.

b) Average Case

Saat diberikan masukan berupa data dengan sebaran Random, program meresponsnya dengan cara yang persis sama. Fungsi rekursif tidak mempedulikan isi dari elemennya, ia murni hanya melihat batas indeks kiri dan kanan. Data acak tersebut tetap akan dibelah dua secara simetris, lalu elemen-elemennya dibandingkan satu per satu saat disatukan kembali di memori utama. Akibat langkah komputasi pembagian dan penggabungan linear yang dijalankan penuh, kompleksitas rata-ratanya juga bernilai $O(N \log N)$.

c) Worst Case

Kondisi memproses himpunan data terurut terbalik (Reverse Sorted) tidak memberikan ancaman berarti untuk algoritma ini. Berbeda dengan Quick Sort yang bisa hancur ketika salah memilih pivot, Merge Sort selalu konsisten membelah memori dari nilai tengah mutlak $(\text{kiri} + \text{kanan}) / 2$. Proses penggabungannya pun tidak memicu lonjakan iterasi berlebih karena pengecekan elemen dilakukan linear. Oleh sebab itu, kompleksitas waktu di skenario paling buruk sekalipun tetap di $O(N \log N)$.

3. Analisis Operasi

a) Jumlah comparison

Pada distribusi (*Sorted* dan *Reverse Sorted*), metrik ini berada di rentang angka yang lebih rendah, yaitu 356-315 kali. Perbedaan jumlah perbandingannya tidak terlalu jauh antar distribusi datanya.

b) Jumlah swap

Tercatat mutlak 0 (Tidak ada). Pemindahan elemen ke *array* utama menggunakan skema penugasan penempatan indeks secara langsung (`data[k] = arr_kiri[i];`), bukan operasi pertukaran `std::swap()`.

c) Jumlah shift

Tercatat 0 (Tidak ada). Tidak ada pemindahan elemen seperti yang ada pada Insertion Sort.

d) Jumlah recursion call

Karena skema pembelahannya yang selalu terbagi dua secara merata terlepas dari bentuk datanya, jumlah pemanggilan fungsinya sangat statis. Baik dihadapkan pada data *Sorted*, maupun data *Reverse Sorted*, nilai `m.recursive_calls` tidak bergeser dari angka 199 kali.

e) Jumlah array accessed

Tercatat 0 (Tidak dihitung). Hitungan akses struktur *array* ditiadakan.

f) Pengaruh bentuk data terhadap jumlah operasi

Algoritma kelas *Divide and Conquer* ini sangat kebal terhadap pengaruh bentuk distribusi awal data. Karena struktur pohon rekursifnya tidak berubah, bentuk data (sudah urut atau berantakan) sama sekali tidak mampu mempengaruhi jumlah

panggilan rekursinya dan hanya sedikit saja memberikan imbas pada jumlah perbandingan di tahap *merge*.

4. Analisis Performa

a) Perbandingan waktu eksekusi antar algoritma

Pada saat diuji *dataset* berukuran masif dengan distribusi yang acak, Merge Sort terbukti berada di tingkat superioritas yang berbeda. Ia membabat habis waktu eksekusi jauh di bawah durasi seluruh algoritma $O(N^2)$ (Bubble, Selection, Insertion) karena sifat eksponensial pemecahan komputasinya.

b) Performa pada dataset kecil dan besar

Di ukuran *dataset* yang sangat minim ($N=100$), algoritma ini justru terlihat agak tertinggal dengan waktu 0.046400 ms, kalah responsif dibanding Insertion Sort karena adanya beban berat dari *overhead* pemanggilan sistem rekursi. Akan tetapi, dominasinya terlihat ketika beban data diset menjadi sangat masif ($N=10000$), di mana ia tetap stabil mencatatkan waktu super cepat di kisaran 3.745800 ms.

c) Pengaruh distribusi data terhadap performa algoritma

Durasi hasil akhir eksekusi antar distribusi data memperlihatkan waktu yang sangat stagnan. Baik ketika merapikan data yang masih *Random*, maupun data yang terbalik (*Reverse Sorted*), waktu pemrosesannya berdekatan secara merata, menjadikannya algoritma paling tangguh secara konsistensi durasi.

d) Hubungan teori kompleksitas dengan hasil eksperimen

Hasil pemantauan menyajikan bukti dari teori Big O Notation untuk $O(N \log N)$. Konsistensi angka statis pada metrik *recursion calls* memberikan fakta bahwa struktur algoritma ini dipastikan selalu identik di setiap skenario. Pertumbuhan beban eksekusi waktu dari angka N yang kecil menuju angka masif juga terbukti tumbuh logaritmik dan tidak meledak menjadi eksponensial.

Quick Sort

A. Source Code

Tabel 5. Source Code Quick Sort

```
136 // Helper Quick Sort
137 int partisi(std::vector<int>& data, int low, int high,
Metrics& m) {
138     int pivot = data[high];
139     int i = (low - 1);
140
141     for (int j = low; j < high; j++) {
142         m.comparisons++;
143         if (data[j] < pivot) {
144             i++;
145             std::swap(data[i], data[j]);
146             m.swaps++;
147         }
148     }
149     std::swap(data[i + 1], data[high]);
150     m.swaps++;
151     return (i + 1);
152 }
153
154 // Helper Quick Sort
155 void rekursif_quick(std::vector<int>& data, int low, int
high, Metrics& m) {
156     m.recursive_calls++;
157     if (low < high) {
158         int titik_partisi = partisi(data, low, high, m);
159         rekursif_quick(data, low, titik_partisi - 1, m);
160         rekursif_quick(data, titik_partisi + 1, high,
m);
161     }
162 }
163
164 // Algoritma Quick Sort
165 void quick_sort(std::vector<int>& data, Metrics& m) {
166     auto start = Clock::now();
167
168     if (data.size() > 1) {
169         rekursif_quick(data, 0, data.size() - 1, m);
```

```

170     }
171
172     m.time_ms      =      std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
173 }

```

B. Output Program

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	678	394	0	129	0	0.005200
Quick Sort	Sorted	100	4950	5049	0	199	0	0.013700
Quick Sort	Reverse Sorted	100	4950	2549	0	199	0	0.011200
Quick Sort	Nearly Sorted	100	2638	2607	0	191	0	0.008300
Quick Sort	Many Duplicates	100	909	236	0	181	0	0.008700
Quick Sort	Random	1000	10537	6125	0	1341	0	0.070000
Quick Sort	Sorted	1000	499500	500499	0	1999	0	1.229400
Quick Sort	Reverse Sorted	1000	499500	250499	0	1999	0	0.790400
Quick Sort	Nearly Sorted	1000	35189	25081	0	1611	0	0.087100
Quick Sort	Many Duplicates	1000	12685	4920	0	1801	0	0.056100
Quick Sort	Random	10000	147606	77996	0	13315	0	0.801400
Quick Sort	Sorted	10000	49995000	50004999	0	19999	0	140.336200
Quick Sort	Reverse Sorted	10000	49995000	25004999	0	19999	0	81.051800
Quick Sort	Nearly Sorted	10000	622473	458547	0	16511	0	1.412900
Quick Sort	Many Duplicates	10000	181016	67083	0	18003	0	0.951300

Gambar 5. Screenshot Output Quick Sort

C. Pembahasan

1. Karakteristik Algoritma

a) Mekanisme Utama

Pertama, melakukan pemanggilan fungsi utama yang memicu rekursif_quick. Karena yang diinginkan adalah memecah urutan berdasarkan satu angka sentral, maka pakai mekanisme pemanggilan `partisi()` yang akan mengunci satu elemen di ujung paling kanan sebagai pusat acuan (`int pivot = data[atas]`). Kemudian, disiapkan indeks bantuan `i` untuk mencatat batas area angka-angka kecil. Untuk memilah elemen sisanya, dipakai perulangan `for` dari kiri ke kanan. Lalu, kemudian baca nilainya. Jika nilai `data[j]` ternyata nilainya kurang dari `pivot`, batas area kecilnya (`i`) diperlebar, lalu nilai tersebut langsung dilempar masuk ke dalam area kecil itu pakai `std::swap()`. Kalau tidak lebih kecil, lewati saja. Setelah penelusuran data berakhir, maka tersisa satu langkah wajib, pindahkan nilai `pivot` yang masih di ujung tadi ke tepat di sebelah kanan indeks `i` (titik pemisah), lalu kembalikan posisi indeks pemisah tersebut. Hal ini diulang terus melalui

pemanggilan rekursif untuk sisi kiri dan sisi kanan dari titik pemisah sampai habis tak tersisa.

b) Stabilitas (*Stable*)

Pada eksekusi pertukaran di dalam perulangan `for` saat proses partisi berlangsung.

```
if (data[j] < pivot) {  
    i++;  
    std::swap(data[i], data[j]);  
    m.swaps++;  
}
```

Mekanisme ini menukar (`swap`) nilai yang terdeteksi lebih kecil dari *pivot* ke area depan dengan melompati rentetan angka di tengah-tengahnya. Akibat proses pencabutan paksa dan pelompatan memori ini, stabilitas menjadi hancur. Elemen bernilai kembar yang tadinya memiliki posisi berurutan bisa saling mendahului karena dilempar oleh operasi pertukaran ini. Sehingga, Quick Sort di sini bersifat tidak *stable*.

c) Penggunaan Memori (*In-place*)

Alur komputasinya secara langsung menggeser dan menukar data langsung di dalam batas vector `data` asalnya. Kode ini tidak mendeklarasikan dan mengalokasikan memori array bantuan sama sekali, sehingga algoritma ini teratas dari sisi efisiensi kompleksitas ruang memori di tingkat $O(1)$. Sehingga, algoritma ini tergolong *in-place*.

2. Kompleksitas Waktu

a) Best Case

Dalam implementasi kodenya, algoritma Quick sort ini bekerja dengan memilih satu elemen sebagai titik pusat dan membagi sisanya menjadi dua bagian.

```
int titik_partisi = partisi(data, low, high, m);  
rekursif_quick(data, low, titik_partisi - 1, m);  
rekursif_quick(data, titik_partisi + 1, high, m);
```

Ketika dihadapkan pada skenario data yang bentuk atau acakannya memungkinkan fungsi `partisi` secara kebetulan selalu memilih nilai tengah (median) sebagai acuan (*pivot*), array akan terus terbelah menjadi dua bagian yang persis sama besar.

Pembelahan memori yang seimbang sempurna ini hanya membutuhkan rekursi sebesar $\log N$. Karena setiap tingkat rekursi harus menyusuri N elemen untuk perbandingan, maka terbukti bahwa komputasi terbaiknya dapat dicapai di level $O(N \log N)$.

b) Average Case

Pada data *Random*, pemilihan elemen paling ujung kanan sebagai pivot (`int pivot = data[high];`) akan membelah array ke dalam proporsi yang cukup seimbang, meski tidak sempurna di tengah. Kedalaman pohon rekursinya masih tetap terjaga di rentang logaritmik. Oleh karena itu, pada himpunan data rata-rata, kompleksitas waktunya tetap berjalan sangat cepat pada $O(N \log N)$.

c) Worst Case

Kelemahan kode ini terlihat saat memproses data yang dari awal sudah terurut rapi (*Sorted*) maupun terurut terbalik (*Reverse Sorted*). Karena titik pusat selalu dikunci secara statis pada indeks paling kanan (`pivot = data[high]`), maka nilai yang terpilih pastilah nilai yang paling ekstrem (paling besar atau paling kecil). Hal ini membuat fungsi partisi gagal membelah data, ia malah memisahkan 0 elemen di satu sisi dan $N-1$ elemen di sisi lainnya. Akibatnya, pemanggilan `rekursif_quick` membengkok seolah menjadi perulangan biasa yang menyusuri indeks satu per satu dari ujung ke ujung. Dari alur ini, terbukti bahwa kompleksitas terburuknya langsung merosot menjadi $O(N^2)$.

3. Analisis Operasi

a) Jumlah comparison

Pada input data *Random*, angka `m.comparisons++` di dalam iterasi berjalan sangat minim dan efisien di kisaran 678 kali berkat pemecahan partisi. Sebaliknya, saat diuji dengan data *Sorted / Reverse Sorted*, metrik perbandingannya meledak, yakni menyentuh 4950 kali komparasi.

b) Jumlah swap

Karena program selalu mengunci elemen paling kanan sebagai *pivot*, jumlah *swap* justru meledak pada data yang sejak awal sudah terurut (*Sorted*). Pada data terurut, setiap elemen selalu terdeteksi lebih kecil dari *pivot* di ujungnya, sehingga kondisi `if (data[j] < pivot)` selalu terpenuhi dan memicu pertukaran secara

maksimal hingga 5049 kali. Pada data *Reverse Sorted*, kondisinya berselang-seling di setiap partisi sehingga jumlah *swap*-nya terpotong menjadi separuh, yaitu 2549 kali. Sebaliknya, efisiensi penukaran terlihat saat algoritma memproses data *Random*, di mana ia hanya membutuhkan 394 kali *swap* karena *array* terbelah secara proporsional..

c) Jumlah shift

Tercatat mutlak 0 (Tidak ada). Quick Sort mengelola pemindahan memori tanpa menggunakan penimpaan penggeseran elemen (*assignment shift*).

d) Jumlah recursion call

Di kondisi *Random*, pemanggilan ke fungsi dirinya sendiri terekam sangat efisien di angka 129 kali. Tapi, ketika dipaksa mengurutkan memori yang *Sorted* atau *Reverse*, fungsi ini mengalami krisis komputasi dan memanggil dirinya sendiri secara brutal hingga maksimum sebesar 199 kali panggilan tanpa henti.

e) Jumlah array accessed

Tercatat 0 (Tidak dihitung).

f) Pengaruh bentuk data terhadap jumlah operasi

Sensitivitas Quick Sort terhadap derajat distribusi input data adalah yang paling ekstrim di antara seluruh algoritma yang diuji. Bentuk penyebaran angka di dalam *array* akan secara langsung menentukan apakah algoritma ini akan berjalan efisien seperti Merge Sort atau hancur lebur menjadi sekelas Bubble Sort karena pemilihan *pivot* statis yang tidak disengaja.

4. Analisis Performa

a) Perbandingan waktu eksekusi antar algoritma

Dalam pertandingan komputasi di data ukuran masif *Random*, algoritma ini merupakan yang tercepat. Quick Sort seringkali menghasilkan waktu eksekusi yang sedikit lebih kencang bahkan dari Merge Sort, dikarenakan efisiensi *in-place* miliknya yang meniadakan tugas *processor* untuk memindah-mindahkan variabel *vector* di luar *array* utama.

b) Performa pada dataset kecil dan besar

Di ukuran kecil ($N=100$), performanya nyaris tak terlihat dan selesai seketika di durasi 0.005200 ms. Efisiensi algoritma ini semakin terlihat ketika ukuran data

dikalikan masif ($N=10000$), di mana pada kumpulan nilai random ia bisa menyelesaikan dengan waktu fantastis yang hanya berkisar 0.801400 ms saja.

c) Pengaruh distribusi data terhadap performa algoritma

Durasi eksekusi pada algoritma ini berbanding lurus dengan banyak fungsi rekursi dipanggil. Saat memproses kumpulan data *random*, pemilihan *pivot* bisa membelah *array* dengan cukup seimbang, sehingga catatan waktunya sangat efisien dan menjadi yang tercepat. Sebaliknya, saat program diberikan data *Sorted* ataupun *Reverse Sorted*, *pivot* yang selalu dipatok dari indeks paling kanan (`data[high]`) membuat pembelahan memori selalu berat sebelah. Akibat fungsi partisi yang memburuk, waktu eksekusi pada kedua distribusi data ini merosot dan menjadi sangat lambat.

d) Hubungan teori kompleksitas dengan hasil eksperimen

Catatan metrik eksekusi berhasil mengonfirmasi validasi rumusan Big O Notation. Ledakan pada metrik *recursion calls* saat data bernilai *Sorted* menjadi pembuktian di terminal uji coba, bahwa skenario terburuk $O(N^2)$ pada algoritma Quick Sort tanpa adanya perbaikan penentuan *pivot* adalah benar.

Radix Sort

A. Source Code

Tabel 6. Source Code Radix Sort

```
176 // Helper Radix Sort (Counting Sort)
177 void counting_sort(std::vector<int>& data, int exp,
Metrics& m) {
178     int n = data.size();
179     std::vector<int> output(n);
180     int count[10] = {0};
181
182     for (int i = 0; i < n; i++) {
183         int digit = (data[i] / exp) % 10;
184         count[digit]++;
185         m.array_accesses++;
186     }
187
188     for (int i = 1; i < 10; i++) {
189         count[i] += count[i - 1];
190         m.array_accesses += 2;
191     }
192
193     for (int i = n - 1; i >= 0; i--) {
194         int digit = (data[i] / exp) % 10;
195         output[count[digit] - 1] = data[i];
196         count[digit]--;
197         m.array_accesses += 3;
198     }
199
200     for (int i = 0; i < n; i++) {
201         data[i] = output[i];
202         m.array_accesses++;
203     }
204 }
205
206 // Algoritma Radix Sort
207 void radix_sort(std::vector<int>& data, Metrics& m) {
208     if (data.empty()) return;
209     auto start = Clock::now();
210
211     int angka_max = data[0];
```

```

212     for (size_t i = 1; i < data.size(); i++) {
213         if (data[i] > angka_max) {
214             angka_max = data[i];
215         }
216     }
217
218     for (int exp = 1; angka_max / exp > 0; exp *= 10) {
219         counting_sort(data, exp, m);
220     }
221
222     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
223 }

```

B. Output Program

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Radix Sort	Random	100	0	0	0	0	1554	0.005800
Radix Sort	Sorted	100	0	0	0	0	1554	0.005400
Radix Sort	Reverse Sorted	100	0	0	0	0	1554	0.005000
Radix Sort	Nearly Sorted	100	0	0	0	0	1554	0.005300
Radix Sort	Many Duplicates	100	0	0	0	0	1036	0.003600
Radix Sort	Random	1000	0	0	0	0	20072	0.085300
Radix Sort	Sorted	1000	0	0	0	0	20072	0.040100
Radix Sort	Reverse Sorted	1000	0	0	0	0	20072	0.040100
Radix Sort	Nearly Sorted	1000	0	0	0	0	20072	0.040400
Radix Sort	Many Duplicates	1000	0	0	0	0	15054	0.029600
Radix Sort	Random	10000	0	0	0	0	250090	0.552100
Radix Sort	Sorted	10000	0	0	0	0	250090	0.496500
Radix Sort	Reverse Sorted	10000	0	0	0	0	250090	0.498400
Radix Sort	Nearly Sorted	10000	0	0	0	0	250090	0.496400
Radix Sort	Many Duplicates	10000	0	0	0	0	200072	0.380300

Done.
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\sorting> |

Gambar 6. Screenshot Output Radix Sort

C. Pembahasan

1. Karakteristik Algoritma

a) Mekanisme Utama

Pertama, melakukan inisiasi pencarian nilai paling maksimum di dalam data sebagai acuan batas perulangan. Karena yang diinginkan adalah membedah dari digit paling belakang, maka disiapkan variabel multiplier `int exp = 1;`. Lalu perulangan utama traverse berjalan memanggil fungsi `counting_sort`. Di dalam fungsi tersebut, setiap angka diurai digitnya pakai operator modulo (`data[i] / exp`) $\% 10$. Digit yang didapat ini dijadikan indeks untuk menambahkan nilai di wadah

`count[]`. Setelah frekuensi digit terhitung, diubah menjadi nilai kumulatif agar program tahu persis indeks elemen tersebut harus diletakkan di mana. Kemudian dilakukan perulangan mundur, menjatuhkan nilai elemen ke indeks yang tepat di vector `output[]`. Terakhir, salin kembali seluruh isi `output` ke dalam array data yang asli. Perulangan di fungsi utama dikalikan 10 (`exp *= 10`) untuk menggeser ke nilai puluhan, ratusan, seterusnya, sampai nilai `angka_max` habis terbagi.

b) Stabilitas (*Stable*)

Logika penjaga stabilitasnya berada di bagian akhir fungsi helpernya, yaitu

```
for (int i = n - 1; i >= 0; i--) {  
    int digit = (data[i] / exp) % 10;  
    output[count[digit] - 1] = data[i];  
    count[digit]--;  
    ...  
}
```

Karena perulangan untuk menempatkan angka ke wadah `output` dijalankan mundur (`i = n - 1; i >= 0`), elemen kembar yang posisi awalnya berada di sebelah kanan akan diletakkan lebih dulu di indeks paling belakang dalam wadah *array*-nya. Hal ini akan menjamin bahwa angka yang masuk belakangan posisinya tidak akan menimpa dan membalik urutan angka kembar yang ada di depannya. Tanpa ini, hasil pengurutan digit puluhan akan merusak hasil pengurutan digit satuan sebelumnya.. Sehingga, Radix Sort di sini bersifat *stable*.

c) Penggunaan Memori (In-place)

Di fungsi `counting_sort`.

```
std::vector<int> output(n);  
int count[10] = {0};
```

Untuk mengelompokkan elemen sesuai urutan digitnya, program mendeklarasikan array besar baru bernama `output` yang seukuran dengan jumlah data asli (N), ditambah sebuah array perhitungan `count` sebesar basis decimal. Akibat pengalokasian memori ganda di setiap perputaran digitnya ini, ruang memori yang

dibutuhkan sangat besar, menjadikannya algoritma dengan kompleksitas ruang $O(N + k)$. Sehingga, algoritma ini tidak tergolong *in-place*.

2. Kompleksitas Waktu

a) Best Case

Algoritma ini bekerja dengan cara membedah angka digit per digit.

```
for (int exp = 1; angka_max / exp > 0; exp *= 10) {  
    counting_sort(data, exp, m);  
}
```

Meskipun program menerima input data Sorted, program tidak punya cara untuk mengetahuinya. Ia tetap akan mencari angka terbesar (`angka_max`) untuk mengetahui jumlah digit maksimal (d). Lalu perulangan pemotongan digit (`exp *= 10`) akan terus berjalan memanggil fungsi `counting_sort` sebanyak jumlah digit tersebut. Di dalam `counting_sort` itu sendiri, penelusuran struktur data array (N) dan rentang wadah angkanya ($k=10$) selalu berjalan penuh. Oleh karena itu, kompleksitas waktu terbaiknya adalah $O(d*(N + k))$.

b) Average Case

Ketika memproses data Random, mekanisme penelusuran digitnya tidak terpengaruh sama sekali. Selama nilai maksimum di dalam array tersebut tetap sama, program akan melakukan pemecahan ke dalam `count` hitungan secara proporsional dari indeks kiri ke kanan. Karena proses membagi, menghitung, dan menempatkan angkanya bersifat linier, kompleksitas rata-ratanya juga stabil di angka $O(d*(N + k))$.

c) Worst Case

Kondisi data Reverse Sorted tidak memicu masalah apapun seperti sort yang lain. Program mengabaikan urutan awal elemen dan hanya melihat elemen dari struktur nilai belakangnya (satuan, puluhan, ratusan). Seluruh perulangan `for` untuk mengurai dan menyatukan array selalu tereksekusi secara konstan. Maka, kompleksitas waktu pada keadaan paling buruk sekalipun tidak akan bergeser dari $O(d * (N + k))$.

3. Analisis Operasi

a) Jumlah comparison

Tercatat 0 (Tidak ada). Ini adalah satu-satunya algoritma penyortiran di praktikum ini yang tidak memakai logika membanding-bandingkan elemen satu sama lain ($>$ atau $<$) untuk memindahkan data.

b) Jumlah swap

Tercatat 0 (Tidak ada). Penempatan urutan dilempar menggunakan penugasan indeks langsung menuju wadah `output`, bukan dari metode `swap` (`std::swap`).

c) Jumlah shift

Tercatat 0 (Tidak ada).

d) Jumlah recursion call

Tercatat 0 (Tidak ada). Seluruh strukturnya menggunakan perulangan iteratif biasa *for-loop*.

e) Jumlah array accessed

Ini adalah metrik pengganti *comparison* untuk Radix Sort. Karena logika utamanya pada seberapa sering program membuka dan memanipulasi slot memori, kode `m.array_accesses++` ditempatkan di setiap pembacaan nilai *array*. Terlepas dari apakah datanya *Sorted* atau *Reverse*, besaran metrik pencatatan operasi memori ini selalu konsisten dan mentok di titik angka kisaran 1554 kali akses.

f) Pengaruh bentuk data terhadap jumlah operasi

Berbeda dari algoritma lain, derajat distribusi data di sini memiliki pengaruh. Satu-satunya hal pada data yang memengaruhi panjang pendeknya proses komputasi operasi algoritma ini hanyalah jumlah banyaknya digit penyusun angka terbesarnya. Semakin panjang digit angkanya, iterasi `counting_sort`-nya akan semakin sering dipanggil.

4. Analisis Performa

a) Perbandingan waktu eksekusi antar algoritma

Dalam pertandingan komputasi data berskala besar, Radix Sort mendominasi dari segi kecepatan. Karena ia tidak terikat batasan $O(N \log N)$ milik algoritma

pembandingan, Radix Sort mengungguli Merge Sort dan Bubble Sort, dengan mengorbankan alokasi ruang memori fisik.

b) Performa pada dataset kecil dan besar

Di skala populasi data kecil ($N=100$), performanya di terminal menghasilkan waktu 0.005800 ms, beratnya *overhead* waktu untuk berkali-kali mencetak *vector array* helper dari nol. Akan tetapi, performanya cocok untuk himpunan data besar ($N=10000$), di mana ia dengan cepat menyelesaikan penyusunan dengan hanya berkisar 0.552100 ms.

c) Pengaruh distribusi data terhadap performa algoritma

Rentang selisih durasi antara pemrosesan data berbentuk *Sorted*, *Random*, hingga *Reverse Sorted* cukup tipis. Hal ini mengonfirmasi bahwa selama rentang besaran angka maksimumnya masih sama, *processor* mengeksekusi beban muatan kerja yang setara pada keadaan urutan data yang bagaimanapun.

d) Hubungan teori kompleksitas dengan hasil eksperimen

Hasil keseluruhan metrik menjadi bukti kepastian teori kompleksitas $O(d*(N+k))$. Hasilnya menunjukkan bahwa catatan `m.array_accesses` bergantung pada pengali d (jumlah eksponen digit maksimal), dan tidak ada lonjakan eksponensial maupun degradasi menjadi fungsi kuadratik saat susunan data masukannya diacak separah apapun.

TAUTAN GITHUB

<https://github.com/DSA26-ULM/module-4-sorting-nullrym>